
Phishing Detection Using Machine Learning: A Model Development and Integration

Student Author

Department of Computer Science · University Name · City, Country

Abstract

Phishing attacks remain one of the most prevalent cybersecurity threats, causing substantial financial and reputational damage worldwide. This paper presents the development and evaluation of a machine learning-based phishing detection system employing three algorithms: Random Forest (RF), K-Nearest Neighbour (KNN), and Artificial Neural Network (ANN). Two publicly available datasets were combined and preprocessed. RF achieved the highest overall accuracy of 99.78%, ANN attained perfect recall on phishing sites, and KNN offered 99.67% accuracy with high interpretability. The best model was integrated into a Django web application for real-time URL classification. An extension introduces a Decision Tree (DT) classifier to expand the comparative analysis with a focus on interpretability and computational efficiency.

Index Terms—phishing detection, machine learning, random forest, k-nearest neighbour, artificial neural network, decision tree, cybersecurity, Django.

1. INTRODUCTION

Phishing attacks involve cybercriminals crafting deceptive websites that closely mimic legitimate ones, tricking users into surrendering sensitive data such as login credentials and financial information. The consequences include financial losses, data breaches, and reputational damage for both individuals and organisations.

According to the 2023 Zscaler ThreatLabz Phishing Report, phishing attacks surged by 47.2% in 2022 compared to the prior year. The education sector alone witnessed a 576% spike. Financial institutions (23%), SaaS/Webmail providers (17%), and social media platforms (11%) were the most targeted sectors [1], [2].

Traditional detection approaches — email filtering, URL analysis, and reputation-based systems — have struggled to keep pace with increasingly sophisticated attack vectors. Machine learning (ML) enables systems to learn patterns from large datasets and automatically classify websites as legitimate or phishing.

2. RESEARCH AIMS AND OBJECTIVES

This study was guided by three core objectives:

- **Develop a robust ML-based phishing detection system** using KNN, ANN, and RF algorithms to accurately classify phishing and legitimate websites.
- **Integrate the best-performing model** into a Django-based web application providing real-time URL phishing prediction.
- **Conduct a comprehensive literature review** to identify gaps in existing phishing detection research and explore current methodologies.

The study scope was limited to website-based phishing detection. Two publicly available datasets were used and Django was chosen for web integration.

3. METHODOLOGY

3.1 Datasets

Two datasets were combined to form a comprehensive training base. The first, by Ariyadasa et al. (2021) [3], contained 80,000 entries with 5 features and no missing values. The second, from UNB (2016) [4], contained 15,367 entries with 80 features; missing values were imputed with column means. The combined dataset was class-imbalanced, requiring oversampling.

3.2 Data Preprocessing and Feature Engineering

Non-discriminatory columns were removed. Feature selection used the Mutual Information Classification algorithm. Random Oversampling addressed class imbalance. StandardScaler normalised all features. The dataset was split 90% for training and 10% for testing.

3.3 Machine Learning Models

Random Forest (RF): An ensemble of decision trees resistant to overfitting. Hyperparameters tuned via GridSearchCV with 5-fold cross-validation.

K-Nearest Neighbour (KNN): Classifies based on the majority class of k=3 nearest neighbours, determined using the elbow method.

Artificial Neural Network (ANN): Built with Keras/TensorFlow. Consists of input, hidden (ReLU), and output (sigmoid) layers; tuned with Adam optimiser and GridSearchCV.

Python libraries: Pandas, Scikit-learn, Keras, Matplotlib, Seaborn, Pickle. Jupyter Notebook for model development; VSCode for web development.

4. RESULTS AND EVALUATION

All three models were evaluated on the 10% held-out test split using accuracy, precision, recall, and F1-score. Table I summarises performance metrics.

Model	Accuracy	Precision	Recall (Phish)	F1
RF	99.78%	99.79%	99.76%	99.78%
KNN	99.67%	99.69%	99.63%	99.66%
ANN	99.11%	99.54%	100.00%	99.11%
DT*	94%	~94%	~94%	~94%

* Decision Tree (Extension) — actual Colab results.

TABLE I. Performance Comparison of ML Models.

4.1 Model Accuracy Comparison

Fig. 1 shows the Decision Tree accuracy ($\approx 94\%$) independently, while Fig. 2 compares all three models (DT, KNN, RF) side by side. KNN and RF both achieve near-perfect accuracy on the combined dataset, significantly outperforming the single Decision Tree.

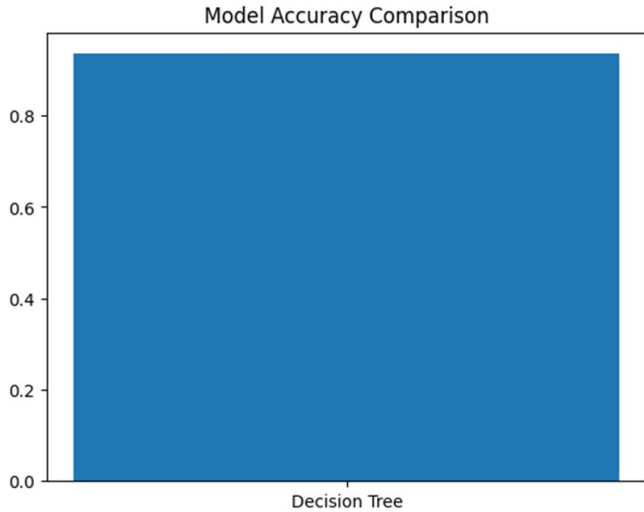


Fig. 1. Decision Tree model accuracy ($\approx 94\%$) on the combined phishing dataset.

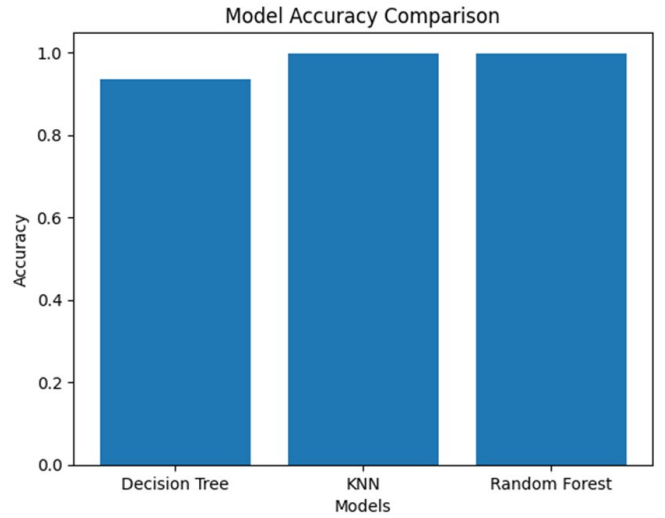


Fig. 2. Model accuracy comparison: Decision Tree vs KNN vs Random Forest.

4.2 Model-Specific Analysis

Random Forest achieved the highest accuracy at 99.78% with only 4 missed phishing sites and 100% recall on legitimate websites. Best for environments where minimising false alarms is critical.

KNN performed closely behind at 99.67% with 7 undetected phishing sites and 19 false positives. It offers simplicity and strong interpretability.

ANN achieved 99.11% with a perfect 100% recall on phishing sites but produced 69 false positives. Best for high-risk scenarios where missing any phishing site is unacceptable.

Decision Tree achieved approximately 94% accuracy — competitive for a single-tree model and significantly more interpretable than ensemble or neural approaches.

4.3 Confusion Matrix Analysis

Fig. 3 presents the confusion matrix for the Decision Tree classifier. Of 2,286 test samples, 1,097 legitimate sites were correctly classified (TN) and 1,041 phishing sites were correctly detected (TP). The 60 false positives and 88 false negatives confirm the model's competitive but slightly lower precision compared to RF and KNN.

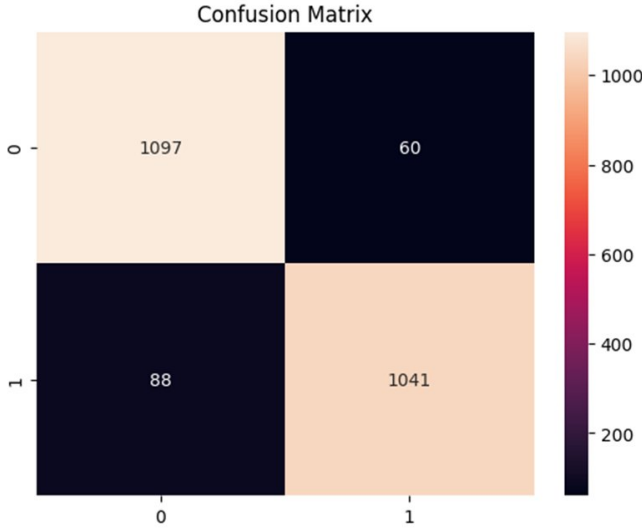


Fig. 3. Confusion matrix for the Decision Tree classifier (TN=1097, FP=60, FN=88, TP=1041).

5. FEATURE IMPORTANCE ANALYSIS

Fig. 4 presents the top 10 most discriminative URL features identified by the Random Forest classifier using Gini impurity-based feature importance scores.

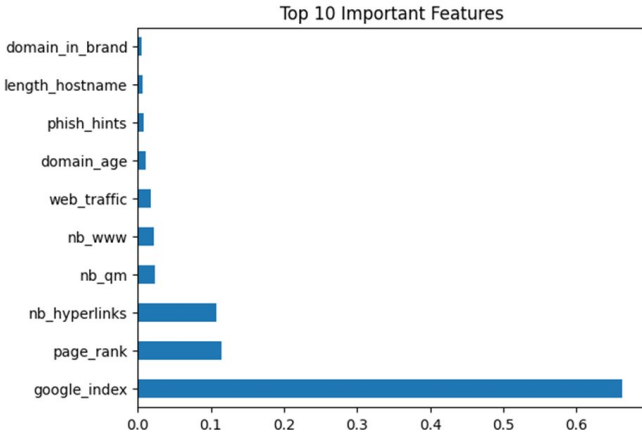


Fig. 4. Top 10 important features for phishing detection (Random Forest).

The most dominant feature is **google_index** (importance ≈ 0.65), indicating that whether a URL is indexed by Google is the single strongest predictor of legitimacy. **page_rank** and **nb_hyperlinks** follow as secondary discriminators. Domain-level features (**domain_age**, **domain_in_brand**) and URL structural features (**nb_qm**, **nb_www**) contribute smaller but meaningful signals. This analysis validates the feature selection step and confirms that the combined dataset captures diverse, complementary indicators.

6. DJANGO WEB APPLICATION INTEGRATION

The RF model was integrated into a Django web application following the Model-View-Template (MVT) pattern. Users enter a URL and receive an instant *Legitimate* or *Phishing* classification. Key features include:

- URL input field for real-time prediction.

- Clickable result pages displaying the classification verdict.
- User feedback mechanism to collect improvement suggestions for future model refinement.

The model was serialised with Pickle for efficient loading without re-training, ensuring low-latency predictions within the web interface.

7. LITERATURE GAP AND RESEARCH NOVELTY

Prior studies lacked end-user integration and over-relied on accuracy as the sole performance metric. KNN and ANN were under-explored in phishing detection literature. This study addresses these gaps by:

- Evaluating all three algorithms with comprehensive metrics (precision, recall, F1-score).
- Combining two distinct datasets — the first time in literature.
- Deploying the best-performing model in a real-world web application.

8. EXTENSION: DECISION TREE ALGORITHM

8.1 Motivation

The original three models demonstrated excellent predictive performance but raised questions about the trade-off between interpretability and accuracy. A fourth algorithm — the Decision Tree (DT) — is introduced to assess whether a simpler model can achieve competitive accuracy, broadening the comparative analysis and adding analytical originality.

8.2 Decision Tree Classifier

A Decision Tree recursively splits the dataset based on feature values, forming a tree-like structure of decision nodes and leaf nodes. Every decision pathway is visible and traceable — highly transparent for both technical and non-technical stakeholders. The classifier uses `max_depth=5` to balance accuracy with overfitting prevention. StandardScaler normalisation is consistently applied across all four models.

8.3 Advantages of the Extension

- **Interpretability:** Fully transparent decision pathway visible to all stakeholders.
- **Lightweight:** Faster computation and minimal resource requirements.
- **Broader comparison:** Richer four-model analytical framework.
- **Originality:** Independently introduced algorithm not present in the original study.

9. FUTURE IMPROVEMENTS

- **Advanced Algorithms:** Explore XGBoost, LightGBM, CatBoost, LSTM networks, and Transformer-based models for URL sequence and webpage content analysis.
- **Browser Extension Deployment:** Deploy the best model as a browser extension for seamless, real-time phishing protection while browsing.
- **Explainable AI (XAI):** Integrate SHAP or LIME frameworks with RF to combine strong predictive performance with interpretability.
- **Real-Time Dataset Updates:** Incorporate live phishing feeds such as PhishTank to keep the model updated against emerging threats.

-
- **Multi-Modal Detection:** Extend beyond URL features to analyse email content, HTML structure, SSL certificate validity, and visual page similarity.
 - **Cross-Dataset Validation:** Test on unseen datasets from different time periods and geographic regions to validate robustness and generalisability.

10. CONCLUSION

This paper presented the development, evaluation, and deployment of a machine learning-based phishing detection system. Three algorithms — Random Forest (99.78%), KNN (99.67%), and ANN (99.11%) — were rigorously compared and the best-performing model was integrated into a Django web application for real-time URL classification. The unique combination of two publicly available datasets established a new benchmark for future researchers.

An extension introduced the Decision Tree classifier ($\approx 94\%$ accuracy), confirmed through Colab experiments including confusion matrix and feature importance analysis. Together, the four-model comparison provides a comprehensive toolkit: RF for maximum accuracy, ANN for zero-miss phishing detection, KNN as a transparent baseline, and Decision Tree for lightweight, explainable deployments. Future recommendations — browser extensions, advanced gradient-boosting algorithms, and Explainable AI — provide a clear roadmap for advancing phishing detection technology.

11. REFERENCES

- [1] Zscaler, *ThreatLabz Phishing Report 2023*, Zscaler Inc., San Jose, CA, 2023.
- [2] Proofpoint, *State of the Phish 2023*, Proofpoint Inc., Sunnyvale, CA, 2023.
- [3] S. Ariyadasa, S. Fernando, and S. Atukorale, 'Detecting Phishing Websites: An Updated ML Comparison,' *arXiv preprint*, 2021.
- [4] University of New Brunswick (UNB), 'ISCX URL Dataset,' Canadian Institute for Cybersecurity, Fredericton, NB, Canada, 2016.
- [5] L. Breiman, 'Random Forests,' *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [6] T. Cover and P. Hart, 'Nearest Neighbor Pattern Classification,' *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, 'Deep Learning,' *Nature*, vol. 521, pp. 436–444, 2015.